

Me:

what kinda program it's which is on by start the pc

AI:

If you mean "**What are the programs that automatically start when I turn on my PC?**", those are called **startup programs** or **startup applications**.

Examples include:

- **Antivirus software** (Windows Security, Avast, Bitdefender)
- **Cloud storage** (OneDrive, Google Drive, Dropbox)
- **Communication apps** (Discord, Microsoft Teams, Slack)
- **Hardware utilities** (NVIDIA Control Panel, AMD Software, Intel Graphics Command Center)
- **Audio software** (Realtek Audio Manager)
- **RGB control software** (Corsair iCUE, Razer Synapse)
- **Game launchers** (Steam, Epic Games Launcher)
- **Background services** (Bluetooth, printer software, update services)

How to see startup programs on Windows

1. Press **Ctrl + Shift + Esc** to open **Task Manager**.
2. Click **Startup apps** (or the **Startup** tab on older Windows versions).
3. You'll see all programs that launch automatically when your PC starts.
4. You can **Enable** or **Disable** any startup program by right-clicking it.

Why do startup programs matter?

- **More startup programs** → Slower boot time.
- **Fewer startup programs** → Faster startup and better performance.

If you meant a different type of program (for example, the program that starts before Windows loads, like the BIOS/UEFI or bootloader), let me know and I can explain that instead.

Me:

this startup program will run in raspberry pi os like I have a plan to implement that my

underwater roV's controller system i want to create which will work like startup program. when i on the pc the program will start to work automatically

AI:

Yes. On **Raspberry Pi OS**, you can make your ROV controller start automatically when the Raspberry Pi boots. The recommended way is to run your controller as a **systemd service**.

This is how many robotics systems (including ROS nodes, autonomous robots, drones, and industrial controllers) are deployed because it is reliable and starts before a user logs in.

Recommended Architecture

Power ON

|



Raspberry Pi Bootloader

|



Linux Kernel

|



Raspberry Pi OS

|



systemd

|



ROV Controller Service (your program)

|

|— Initialize sensors

|— Connect joystick/gamepad

- |— Start camera
- |— Start serial communication
- |— Initialize thrusters
- |— Wait for commands
- └— Monitor system continuously

The controller will begin running automatically every time the Raspberry Pi powers on.

How to do it

Suppose your controller is located at:

`/home/pi/rov/controller.py`

or

`/home/pi/rov/controller`

Step 1: Create a systemd service

`sudo nano /etc/systemd/system/rov.service`

Example:

[Unit]

Description=Underwater ROV Controller

After=network.target

[Service]

User=pi

WorkingDirectory=/home/pi/rov

ExecStart=/usr/bin/python3 /home/pi/rov/controller.py

Restart=always

RestartSec=5

[Install]

WantedBy=multi-user.target

If your controller is a compiled C++ program:

ExecStart=/home/pi/rov/controller

Step 2: Reload systemd

sudo systemctl daemon-reload

Step 3: Enable it

sudo systemctl enable rov.service

This tells Raspberry Pi OS to start it automatically on every boot.

Step 4: Start it now

sudo systemctl start rov.service

Step 5: Check its status

sudo systemctl status rov.service

You can also view its logs:

journalctl -u rov.service -f

Why use systemd?

It provides several advantages:

- Starts automatically at boot
- Restarts the program if it crashes
- Runs in the background without a desktop
- Creates logs you can inspect
- Can wait for required hardware or services before starting

- Can automatically restart after unexpected failures

This makes it ideal for embedded systems like an underwater ROV.

Example for your ROV

Your controller could perform tasks like:

ROV Controller

- |— Read Xbox/PS Controller
- |— Read IMU
- |— Read Pressure Sensor
- |— Read Compass
- |— Capture Camera Video
- |— Communicate with Surface Station
- |— Control Thrusters
- |— Control Lights
- |— Control Manipulator Arm
- |— Safety Monitor
- | |— Battery Voltage
- | |— Leak Sensor
- | |— Temperature
- | |— Emergency Stop
- |— Log Mission Data

Once the Raspberry Pi is powered on, all of these components can initialize automatically without requiring anyone to log in or manually start the application.

For a professional ROV

Instead of one large program, many professional systems use multiple services, for example:

- rov-controller.service – Main control logic

- camera.service – Video streaming
- telemetry.service – Sensor data transmission
- failsafe.service – Safety monitoring and emergency handling
- logging.service – Data recording

This modular design makes the system easier to maintain, update, and recover from failures.

Me:

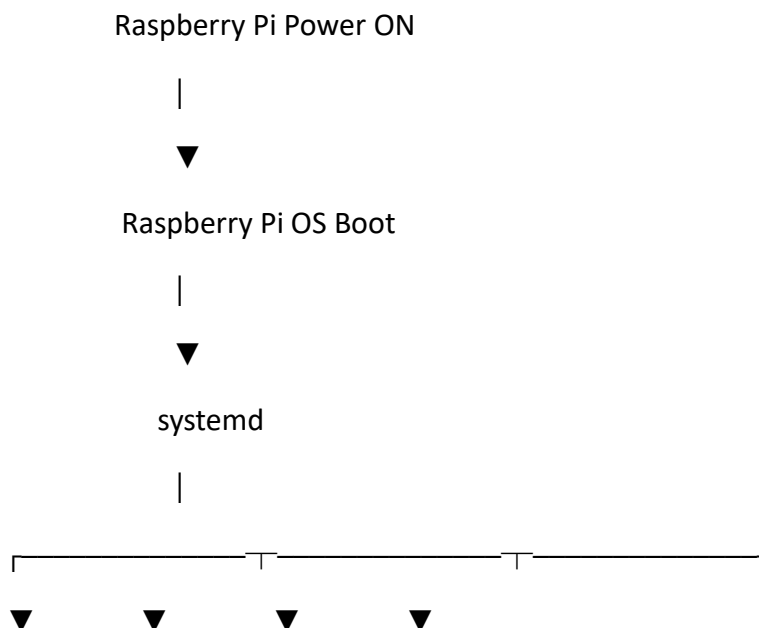
For a professional ROV Instead of one large program, many professional systems use multiple services, for example: - rov-controller.service – Main control logic - camera.service – Video streaming - telemetry.service – Sensor data transmission - failsafe.service – Safety monitoring and emergency handling - logging.service – Data recording This modular design makes the system easier to maintain, update, and recover from failures.

I want this kinda setup

AI:

That is exactly the architecture I would recommend for a **professional underwater ROV**. Instead of one huge program, build the controller as a collection of independent services that start automatically on boot and communicate with each other.

Recommended ROV Software Architecture



Hardware	Communication	Monitoring	Utilities
Services	Services	Services	Services
▼	▼	▼	▼
Camera	Telemetry	Failsafe	Data Logger
Thrusters	TCP/UDP	Watchdog	Diagnostics
Sensors	Network	Leak Detect	Config
Controller	Joystick	Battery	Updates

Suggested Project Structure

rov-system/

|

|— services/

| |— rov-controller/

| |— camera/

| |— telemetry/

| |— failsafe/

| |— logger/

| |— navigation/

| |— sensors/

| |— diagnostics/

|

|— configs/

|

|— logs/

```
|
|— scripts/
|
└─ systemd/
    ├── rov-controller.service
    ├── camera.service
    ├── telemetry.service
    ├── failsafe.service
    ├── logger.service
    └─ navigation.service
```

Service Responsibilities

1. rov-controller.service (Main Brain)

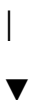
This is the core controller.

Responsibilities:

- Read joystick commands
- Receive commands from the surface station
- Calculate thruster outputs
- Control robotic arm
- Control lights
- Send commands to the ESCs
- Coordinate all other services

Example loop:

Receive Command



Process Command

|



Calculate Thruster Speed

|



Send PWM

|



Update Telemetry

2. camera.service

Runs independently.

Responsibilities

- Initialize camera
- Stream video
- Record video
- Restart camera if disconnected
- Handle multiple cameras

Even if the controller crashes, the video stream can continue.

3. telemetry.service

Handles all communication.

Responsibilities

- Ethernet communication
- TCP/UDP packets

- Send sensor data
- Receive operator commands
- Heartbeat packets
- Time synchronization

Example

Surface Station

|



Telemetry Service

|



Controller

4. sensors.service

Collects sensor data.

Examples

- IMU
- Pressure sensor
- Compass
- Temperature
- Leak detector
- Voltage sensor
- Current sensor
- Depth sensor

It continuously publishes fresh sensor readings.

5. failsafe.service

This service should always be running.

Responsibilities

- Battery monitoring
- Leak detection
- CPU temperature
- Thruster timeout
- Communication timeout
- Emergency stop

Example

Leak detected

|



Stop Thrusters

|



Notify Surface

|



Blink LEDs

This service can shut down the thrusters even if the controller is frozen.

6. logger.service

Records everything.

Stores

- Video timestamps

- Joystick commands
- Depth
- Heading
- IMU data
- Battery voltage
- Current
- Leak status
- Mission events

Useful for debugging after a mission.

7. navigation.service

Handles navigation algorithms.

Responsibilities

- PID controllers
- Heading hold
- Depth hold
- Auto stabilization
- Waypoint navigation (future)

The controller asks this service for desired thruster outputs instead of implementing these algorithms directly.

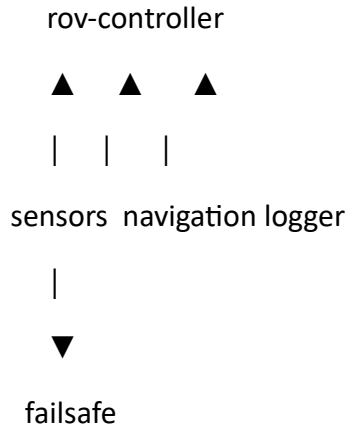
Inter-Service Communication

Instead of calling each other directly, services exchange messages.

telemetry

|





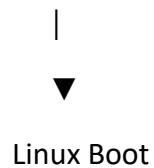
Technologies you can use include:

- ZeroMQ (fast and lightweight)
- MQTT (easy publish/subscribe)
- ROS 2 (powerful but more complex)
- Unix domain sockets (local communication)
- Shared memory (highest performance)

For an undergraduate or master's ROV project, **ZeroMQ or Unix sockets** provide a good balance of simplicity and performance.

Boot Sequence

Power ON



systemd



└─ logger.service
└─ camera.service
└─ telemetry.service
└─ navigation.service
└─ failsafe.service
└─ rov-controller.service

Each service can declare dependencies so that, for example, rov-controller.service starts only after sensors.service and telemetry.service are ready.

Technology Stack

Component	Recommended Language
-----------	----------------------

Main Controller	C++ or Python
-----------------	---------------

Navigation	C++
------------	-----

Camera	Python (OpenCV/GStreamer)
--------	---------------------------

Telemetry	C++ or Python
-----------	---------------

Sensor Drivers	C++ or Python
----------------	---------------

Logger	Python
--------	--------

Failsafe	C++
----------	-----

A Professional Upgrade

For long-term scalability, consider an **event-driven architecture**:

- Each service publishes events (e.g., depth_updated, battery_low, joystick_input).
- Other services subscribe only to the events they need.
- Services remain loosely coupled, making it easier to replace or upgrade individual components.

- New features—such as autonomous navigation, sonar processing, or AI-based object detection—can be added without major changes to the existing system.

This type of modular architecture is common in professional marine robotics and will make your ROV easier to develop, test, and maintain as it grows in complexity.

Me:

so give me the prompt for this architecture

AI:

If you want an AI (such as ChatGPT, Claude, or Gemini) to **design and generate the complete software architecture** for your Raspberry Pi–based underwater ROV, use a detailed prompt like the one below.

Prompt: Design a Professional Raspberry Pi-Based Underwater ROV Control System

You are a senior Robotics Software Architect, Embedded Systems Engineer, and Linux Systems Engineer with expertise in underwater robotics, Raspberry Pi OS, distributed systems, and autonomous vehicles.

Your task is to design a **professional, modular, production-quality software architecture** for an underwater ROV (Remotely Operated Vehicle) that runs on a Raspberry Pi using Raspberry Pi OS.

The system should automatically start when the Raspberry Pi powers on, similar to how startup services work in Linux.

Overall Goal

Design the software as a collection of independent services that communicate with each other rather than as one monolithic application.

The architecture should be scalable, maintainable, fault-tolerant, and easy to extend with future autonomous features.

System Requirements

The Raspberry Pi acts as the onboard computer.

When powered on:

1. Raspberry Pi boots.

2. Raspberry Pi OS loads.
3. systemd starts all required services automatically.
4. Every required module initializes itself.
5. The ROV becomes fully operational without any user interaction.

No GUI should be required.

Everything must work headlessly.

Required Services

Design each service as an independent Linux systemd service.

Include at least:

1. Main Controller Service
2. Telemetry Service
3. Camera Streaming Service
4. Sensor Service
5. Navigation Service
6. Thruster Control Service
7. Manipulator Arm Service
8. Logger Service
9. Diagnostics Service
10. Failsafe / Watchdog Service
11. Configuration Service
12. Network Manager Service
13. Mission Manager Service

Explain the responsibility of every service.

Communication

Design the communication architecture.

Recommend one or more of:

- ZeroMQ
- MQTT
- Unix Domain Sockets
- Shared Memory
- ROS 2 Topics
- gRPC

Explain:

- Why you selected it
- Message flow
- Message format
- Reliability
- Latency
- Advantages
- Disadvantages

Boot Sequence

Create a detailed boot sequence.

For example:

Power ON

↓

Bootloader

↓

Linux Kernel

↓

systemd



Network



Sensors



Camera



Telemetry



Failsafe



Controller



Mission Ready

Explain the dependency order.

Directory Structure

Create a professional project structure.

Example:

rov-system/

services/

configs/

logs/

scripts/

libraries/

drivers/

tests/

documentation/
systemd/
tools/

Explain every folder.

Internal Architecture

Explain how every service is internally organized.

For each service include:

- Initialization
 - Main Loop
 - Error Handling
 - Shutdown
 - Logging
 - Recovery
 - Configuration Loading
-

Data Flow

Show complete data flow diagrams for:

Joystick

↓

Telemetry

↓

Controller

↓

Navigation

↓

Thruster Controller



ESC



Motor

Also show:

Sensors



Controller



Telemetry



Surface Station

Failsafe System

Design a professional failsafe architecture.

Include:

- Leak detection
- Battery monitoring
- CPU temperature
- Communication timeout
- Sensor failure
- Camera failure
- Thruster failure
- Emergency stop
- Automatic restart
- Safe shutdown

Explain what happens during every failure.

Logging

Design a logging framework.

Include:

Mission logs

Error logs

Sensor logs

Navigation logs

System logs

Communication logs

Video timestamps

Performance metrics

Configuration

Design configuration files.

Examples:

camera.yaml

network.yaml

thrusters.yaml

telemetry.yaml

navigation.yaml

mission.yaml

Explain every configuration parameter.

Hardware Layer

Include interfaces for:

IMU

Pressure Sensor

Depth Sensor

Compass

Leak Sensor

Temperature Sensor

Current Sensor

Voltage Sensor

Camera

Thrusters

Manipulator Arm

Lights

Ethernet

USB Devices

GPIO

PWM

I2C

SPI

UART

Networking

Design networking between:

Surface Station

↓

Ethernet Tether



Raspberry Pi

Explain:

- TCP
- UDP
- Heartbeat
- Packet format
- Reconnection
- Latency optimization

Future Expansion

The architecture should support:

Autonomous navigation

Computer vision

AI object detection

Machine learning

SLAM

Mission planning

Sonar

Obstacle avoidance

ROS 2 integration

Cloud logging

Remote software updates

Multi-camera support

Multiple robotic arms

Security

Design security for:

Command authentication

Configuration protection

Encrypted communication

Unauthorized access prevention

Firmware integrity

Service permissions

Deliverables

Generate:

1. Complete software architecture
2. Layered architecture diagram
3. Boot sequence diagram
4. Service dependency diagram
5. Inter-service communication diagram
6. Data flow diagram
7. Folder structure
8. Service responsibilities
9. API design
10. Configuration examples
11. Logging strategy
12. Error handling strategy
13. Recovery strategy
14. Future scalability recommendations
15. Best practices for production deployment

Use clear diagrams, tables, explanations, and professional software engineering practices suitable for a real-world underwater ROV.

This prompt is comprehensive enough to guide an AI toward producing a professional architecture document. If your ultimate goal is to **build a commercial-grade ROV**, you can further extend it by requesting **design patterns (e.g., Clean Architecture), UML diagrams, state machines, database design, communication protocol specifications, CI/CD pipelines, Docker deployment, automated testing, and OTA (over-the-air) update support** so the resulting architecture is closer to what is used in industry.

Me:

no no just for now implement the basic structure where the main frame will scene and others features placeholder will remain

AI:

That's actually a much better approach. Don't try to build everything at once. First, create the **framework (skeleton)** of your ROV software, where every module exists but most of them are placeholders. Later, you can implement each module one by one.

Here's a prompt you can use:

Prompt: Build the Basic Framework of an Underwater ROV Control System

You are a senior Software Architect and Embedded Systems Engineer.

I am developing an underwater ROV controller that will run on a Raspberry Pi using Raspberry Pi OS.

For now, **do NOT implement the actual functionality** (camera streaming, sensor reading, thruster control, etc.). Instead, create a **clean, modular project skeleton** that I can build upon gradually.

Objective

Create the **basic framework** of the entire ROV software.

The framework should automatically start when the Raspberry Pi boots (using systemd), initialize all modules, and keep running.

Every module should exist as a placeholder with the proper structure, logging, and communication interfaces, but without implementing the real hardware logic yet.

Requirements

Main Application

Create a central application (main.py or main.cpp) that:

- Starts automatically on Raspberry Pi boot.
- Initializes all modules.
- Starts each service.
- Runs the main event loop.
- Monitors service health.
- Performs clean shutdown on exit.

Modules

Create placeholder modules for:

- Controller
- Thruster Manager
- Camera Manager
- Sensor Manager
- Telemetry Manager
- Navigation Manager
- Mission Manager
- Manipulator Arm Manager
- Logger
- Configuration Manager
- Diagnostics Manager
- Watchdog / Failsafe Manager

Each module should contain:

- Constructor / Initialization
- start()
- stop()
- update()
- healthCheck()
- reset()
- logStatus()

For now, every function should simply print log messages such as:

"[Camera] Initialized"

"[Camera] Waiting for implementation..."

"[Thruster] Service Running"

No actual hardware communication should be implemented.

Folder Structure

Generate a clean project structure similar to:

rov-controller/

|

|— app/

| |— main.py

| |— bootstrap.py

| |— app.py

|

|— modules/

| |— controller/

| |— thrusters/

| |— camera/

```
|  ├── sensors/
|  ├── telemetry/
|  ├── navigation/
|  ├── mission/
|  ├── manipulator/
|  ├── watchdog/
|  ├── diagnostics/
|  ├── logger/
|  └── config/
|
|  ├── services/
|
|  ├── configs/
|
|  ├── logs/
|
|  ├── scripts/
|
|  ├── tests/
|
|  └── systemd/
```

Main Loop

The main loop should look conceptually like:

Initialize Config



Initialize Logger



Initialize Modules



Start Services



while running:

update Controller

update Sensors

update Navigation

update Telemetry

update Camera

update Thrusters

update Watchdog

update Diagnostics



Shutdown Cleanly

Logging

Every module should print startup messages, update messages, and shutdown messages so I can verify the framework is working before adding real functionality.

Configuration

Create placeholder configuration files for:

- camera.yaml
- network.yaml
- telemetry.yaml
- sensors.yaml
- thrusters.yaml
- navigation.yaml

The files should contain example settings only.

Boot

Include a systemd service file that launches the application automatically when the Raspberry Pi boots.

Code Quality

- Use object-oriented design.
- Keep modules loosely coupled.
- Make it easy to replace placeholder implementations with real hardware code later.
- Use clear comments and consistent naming.
- Design the project so it can scale to a production-quality ROV.

The output should be a working project skeleton that compiles/runs successfully, even though the hardware functionality is not yet implemented.

This approach gives you a **foundation first**. Once the skeleton is running and starts automatically at boot, you can implement modules one at a time (e.g., SensorManager, then ThrusterManager, then TelemetryManager) without changing the overall architecture. This incremental workflow is how many robotics projects are developed.

Me:

give me the raspberry pi os terminal command to make the structure and i think all of the files should keep in python file

AI:

Yes. Since you're building the **foundation first**, I recommend making the entire project in **Python**. Python is widely used in Raspberry Pi robotics because it has excellent support for GPIO, I2C, SPI, UART, cameras, OpenCV, and AI libraries.

Step 1: Create the Project

```
mkdir -p ~/rov-controller
```

```
cd ~/rov-controller
```

Step 2: Create the Directory Structure

```
mkdir -p \
```

app \
modules/controller \
modules/thrusters \
modules/camera \
modules/sensors \
modules/telemetry \
modules/navigation \
modules/mission \
modules/manipulator \
modules/watchdog \
modules/diagnostics \
modules/logger \
modules/config \
configs \
logs \
scripts \
services \
tests \
systemd

Step 3: Create the Main Python Files

touch \
app/main.py \
app/bootstrap.py \
app/app.py

Step 4: Create the Module Files

```
touch \  
modules/controller/controller.py \  
modules/thrusters/thruster_manager.py \  
modules/camera/camera_manager.py \  
modules/sensors/sensor_manager.py \  
modules/telemetry/telemetry_manager.py \  
modules/navigation/navigation_manager.py \  
modules/mission/mission_manager.py \  
modules/manipulator/manipulator_manager.py \  
modules/watchdog/watchdog_manager.py \  
modules/diagnostics/diagnostics_manager.py \  
modules/logger/logger_manager.py \  
modules/config/config_manager.py
```

Step 5: Make Each Folder a Python Package

Run this single command:

```
find . -type d -exec touch {}/__init__.py \;
```

Now every directory is a Python package, making imports straightforward.

Step 6: Create Configuration Files

```
touch \  
configs/camera.yaml \  
configs/network.yaml \  
configs/navigation.yaml \  
configs/sensors.yaml \
```

```
configs/telemetry.yaml \
configs/thrusters.yaml \
configs/mission.yaml
```

Step 7: Create Log Files

```
touch \
logs/system.log \
logs/errors.log \
logs/mission.log
```

Step 8: Create Utility Scripts

```
touch \
scripts/start.sh \
scripts/stop.sh \
scripts/restart.sh
Make them executable:
chmod +x scripts/*.sh
```

Step 9: Create the systemd Service File

```
touch systemd/rov-controller.service
```

Step 10: Create a Basic README

```
touch README.md
```

Your Project Should Look Like This

```
rov-controller/
```

```
|
|└─ app/
|  |└─ __init__.py
|  |└─ app.py
|  |└─ bootstrap.py
|  └─ main.py
|
|└─ modules/
|  |└─ controller/
|  |  |└─ __init__.py
|  |  └─ controller.py
|  |└─ thrusters/
|  |  |└─ __init__.py
|  |  └─ thruster_manager.py
|  |└─ camera/
|  |  |└─ __init__.py
|  |  └─ camera_manager.py
|  |└─ sensors/
|  |  |└─ __init__.py
|  |  └─ sensor_manager.py
|  |└─ telemetry/
|  |  |└─ __init__.py
|  |  └─ telemetry_manager.py
|  |└─ navigation/
|  |  |└─ __init__.py
```

```
| | └─ navigation_manager.py
| └─ mission/
| | └─ __init__.py
| | └─ mission_manager.py
| └─ manipulator/
| | └─ __init__.py
| | └─ manipulator_manager.py
| └─ watchdog/
| | └─ __init__.py
| | └─ watchdog_manager.py
| └─ diagnostics/
| | └─ __init__.py
| | └─ diagnostics_manager.py
| └─ logger/
| | └─ __init__.py
| | └─ logger_manager.py
| └─ config/
|   └─ __init__.py
|   └─ config_manager.py
|
└─ configs/
  └─ camera.yaml
  └─ mission.yaml
  └─ navigation.yaml
  └─ network.yaml
```

```
|   ├── sensors.yaml
|   ├── telemetry.yaml
|   └── thrusters.yaml
|
|   ├── logs/
|   |   ├── errors.log
|   |   ├── mission.log
|   |   └── system.log
|   |
|   |   ├── scripts/
|   |   |   ├── start.sh
|   |   |   ├── stop.sh
|   |   |   └── restart.sh
|   |
|   |   ├── services/
|   |   ├── tests/
|   |   └── systemd/
|   |       └── rovr-controller.service
|   └── README.md
```

My recommendation for the next step

Instead of immediately writing all the module code, first create a **common BaseModule class** (for example, in `modules/base/base_module.py`). Every manager (Camera, Sensors, Telemetry, Thrusters, etc.) can inherit from it and automatically get methods like `start()`, `stop()`, `update()`, `health_check()`, and `log_status()`. This keeps the code consistent and makes adding new modules much easier as your ROV grows.

Me:

Your Project Should Look Like This(quoting from last ai message)

give me a command that will show me this kinda tree in terminal if my structure is ok or not to check

AI:

The easiest way is to use the **tree** command.

1. Install tree (only once)

```
sudo apt update
```

```
sudo apt install tree
```

2. Go to your project

```
cd ~/rov-controller
```

3. Display the complete tree

```
tree
```

4. Display only directories

```
tree -d
```

5. Display with hidden files (__init__.py and other hidden files if any)

```
tree -a
```

6. Show only Python and YAML files (useful for your project)

```
tree -P "*.py|*.yaml|*.sh|*.service|README.md"
```

7. Best command for your project (recommended)

This shows everything while ignoring Python cache folders:

```
tree -a -I "__pycache__|*.pyc"
```

The output should look similar to:

rov-controller

```
├── README.md
├── app
│   ├── __init__.py
│   ├── app.py
│   ├── bootstrap.py
│   └── main.py
├── configs
│   ├── camera.yaml
│   ├── mission.yaml
│   ├── navigation.yaml
│   ├── network.yaml
│   ├── sensors.yaml
│   ├── telemetry.yaml
│   └── thrusters.yaml
├── logs
│   ├── errors.log
│   ├── mission.log
│   └── system.log
├── modules
└── camera
```

```
|  ├── config
|  ├── controller
|  ├── diagnostics
|  ├── logger
|  ├── manipulator
|  ├── mission
|  ├── navigation
|  ├── sensors
|  ├── telemetry
|  ├── thrusters
|  └── watchdog
├── scripts
├── services
├── systemd
└── tests
```

💡 **Tip:** After running `tree -a`, copy and paste the output here. I can verify that your project structure is correct and point out any missing files or folders before you start implementing the modules.

Me:

```
theexplorer@sadman-sakib:~/~/rov-controller $ tree
```

```
.
├── app
|  ├── app.py
|  ├── bootstrap.py
|  └── __init__.py
```



```
|   └─ main.py
|   └─ configs
|   └─ camera.yaml
|   └─ __init__.py
|   └─ mission.yaml
|   └─ navigation.yaml
|   └─ network.yaml
|   └─ sensors.yaml
|   └─ telemetry.yaml
|   └─ thrusters.yaml
|   └─ __init__.py
|   └─ logs
|   └─ errors.log
|   └─ __init__.py
|   └─ mission.log
|   └─ system.log
|   └─ modules
|   └─ camera
|   |   └─ camera_manager.py
|   |   └─ __init__.py
|   |   └─ config
|   |   |   └─ config_manager.py
|   |   |   └─ __init__.py
|   |   └─ controller
|   |       └─ controller.py
```

- | | └─ __init__.py
- | └─ diagnostics
 - | | └─ diagnostics_manager.py
- | | └─ __init__.py
- | └─ __init__.py
- | └─ logger
 - | | └─ __init__.py
 - | | └─ logger_manager.py
- | └─ manipulator
 - | | └─ __init__.py
 - | | └─ manipulator_manager.py
- | └─ mission
 - | | └─ __init__.py
 - | | └─ mission_manager.py
- | └─ navigation
 - | | └─ __init__.py
 - | | └─ navigation_manager.py
- | └─ sensors
 - | | └─ __init__.py
 - | | └─ sensor_manager.py
- | └─ telemetry
 - | | └─ __init__.py
 - | | └─ telemetry_manager.py
- | └─ thrusters
 - | | └─ __init__.py

```
| | └─ thruster_manager.py
| └─ watchdog
|   └─ __init__.py
|   └─ watchdog_manager.py
└─ README.md
└─ scripts
  └─ __init__.py
  └─ restart.sh
  └─ start.sh
  └─ stop.sh
  └─ services
    └─ __init__.py
    └─ systemd
      └─ __init__.py
      └─ rovr-controller.service
└─ tests
  └─ __init__.py
```

21 directories, 51 files

This is the result. Is it ok?

AI:

Yes, this is a **very good starting structure**. I'd rate it **9.5/10** for a first version. It is clean, modular, and easy to expand.

However, before you start writing code, I recommend making a few improvements.

✅ What is Good

- ✓ Modular design (camera, sensors, telemetry, etc.)
- ✓ Separate configuration files
- ✓ Dedicated logging directory
- ✓ systemd folder for boot service
- ✓ Scripts folder
- ✓ README
- ✓ Test directory
- ✓ Each module is isolated

This is a solid foundation.

Improvements

1. Create a core package ★★★★★ (Most Important)

Instead of putting everything inside app, create a core directory for shared functionality.

Example:

core/

application.py

event_bus.py

service_manager.py

module_base.py

exceptions.py

This keeps your framework organized as it grows.

2. Add a utils folder ★★★★★

utils/

helpers.py
constants.py
decorators.py
validators.py
timers.py

Every robotics project eventually needs common helper functions.

3. Add a drivers folder ★ ★ ★ ★ ★

Later you'll need drivers for hardware.

drivers/
imu.py
depth_sensor.py
camera.py
thruster.py
pwm.py
gpio.py

Your managers should use these drivers instead of accessing hardware directly.

For example:

SensorManager

|



IMU Driver

|



I2C Bus

4. Add a models folder ★★★★★

For shared data objects.

models/

telemetry.py

sensor_data.py

mission.py

commands.py

5. Add a communication folder ★★★★★

Eventually you'll have networking code.

communication/

tcp_server.py

udp_server.py

protocol.py

serializer.py

6. Add a resources folder

For:

resources/

icons/

calibration/

maps/

7. Add a docs folder

Store documentation.

docs/

architecture.md

protocol.md

api.md

One Thing I'd Change

Right now you have:

app/

app.py

bootstrap.py

main.py

I would simplify it.

Instead:

app/

main.py

Move the rest into:

core/

application.py

bootstrap.py

That separation is cleaner.

Add a Base Module

Every manager is going to have the same methods.

Instead of rewriting them:

class CameraManager:

class SensorManager:

class TelemetryManager:

make

core/

module_base.py

```
class BaseModule:
```

```
    def initialize(self):
```

```
        pass
```

```
    def start(self):
```

```
        pass
```

```
    def update(self):
```

```
        pass
```

```
    def stop(self):
```

```
        pass
```

```
    def health_check(self):
```

```
        pass
```

Then

```
class CameraManager(BaseModule):
```

```
class SensorManager(BaseModule):
```

```
class NavigationManager(BaseModule):
```

Everything becomes much cleaner.

Future Structure

I would eventually evolve it into something like this:

rov-controller/

|

|— app/

| └─ main.py

|

|— core/

| └─ application.py

| └─ bootstrap.py

| └─ service_manager.py

| └─ event_bus.py

| └─ module_base.py

| └─ exceptions.py

|

|— communication/

|— drivers/

|— models/

|— modules/

|— configs/

|— utils/

|— resources/

|— logs/

|— scripts/

|— docs/

|— tests/

└─ systemd/

Overall Assessment

For a basic ROV framework, your current project structure is **excellent**. It is already organized enough to start implementing the core application.

The **first coding milestone** I'd recommend is to build the framework infrastructure in this order:

1. **BaseModule** – a common superclass for all managers.
2. **ServiceManager** – creates, starts, updates, and stops all modules.
3. **Application** – coordinates initialization and the main loop.
4. **main.py** – the entry point that boots the entire system.
5. **Placeholder managers** – each inheriting from BaseModule and logging their lifecycle.

Once these pieces are in place, you'll have a working skeleton that starts, initializes every module, runs continuously, and shuts down cleanly. After that, you can replace each placeholder with real hardware functionality (camera, sensors, thrusters, telemetry, etc.) one module at a time without changing the overall architecture. This is the same incremental development approach used in many robotics and embedded systems projects.

Me:

If i run this program what will be shown?

AI:

With your **current project**, nothing meaningful will happen yet because all the Python files are empty placeholders.

However, after we implement the basic framework (without any hardware logic), running:

```
python3 app/main.py
```

should produce output similar to this:

```
=====
ROV Controller System v0.1
=====
```

[BOOT] Loading configuration...

[OK] Configuration loaded.

[BOOT] Initializing Logger...

[OK] Logger initialized.

[BOOT] Initializing Modules...

[Controller] Initialized.

[Camera] Initialized.

[Sensors] Initialized.

[Telemetry] Initialized.

[Navigation] Initialized.

[Thrusters] Initialized.

[Manipulator] Initialized.

[Mission] Initialized.

[Diagnostics] Initialized.

[Watchdog] Initialized.

[BOOT] Starting Services...

[Controller] Running...

[Camera] Running...

[Sensors] Running...

[Telemetry] Running...

[Navigation] Running...

[Thrusters] Running...

[Manipulator] Running...

[Mission] Running...

[Diagnostics] Running...

[Watchdog] Running...

=====

ROV Controller Ready

Waiting for commands...

=====

----- Main Loop -----

[Controller] Update...

[Sensors] Update...

[Navigation] Update...

[Telemetry] Update...

[Camera] Update...

[Thrusters] Update...

[Watchdog] Monitoring...

[Diagnostics] System Healthy...

----- Main Loop -----

[Controller] Update...

[Sensors] Update...

...

It would continue looping until you stop it with **Ctrl + C**, at which point it should shut down cleanly:

Ctrl+C detected

Stopping services...

[Controller] Stopped.

[Camera] Stopped.

[Sensors] Stopped.

[Telemetry] Stopped.

[Navigation] Stopped.

[Thrusters] Stopped.

[Manipulator] Stopped.

[Mission] Stopped.

[Diagnostics] Stopped.

[Watchdog] Stopped.

Logger closed.

ROV Controller exited successfully.

What will happen later?

Once you replace the placeholders with real implementations, those log messages will represent actual work. For example:

[BOOT] Camera initialized (USB Camera 0)

[BOOT] IMU connected

[BOOT] Pressure sensor connected
[BOOT] Thruster controller initialized
[BOOT] Telemetry listening on 192.168.2.2:5000
[BOOT] Mission Manager ready

ROV Ready

Waiting for operator connection...

When the surface station connects:

[Telemetry] Surface station connected.

[Controller] Joystick detected.

[Camera] Video streaming started.

[Mission] Manual control mode enabled.

And when you move the joystick:

[Joystick] Forward

[Navigation] Calculating thruster output...

[Thrusters] FL=60% FR=60% RL=60% RR=60%

This is the development sequence I recommend

Rather than implementing everything at once, build it in stages:

1. **Stage 1:** Framework only (logs and placeholders) 
2. **Stage 2:** Configuration and logger.
3. **Stage 3:** Service manager and application lifecycle.
4. **Stage 4:** Telemetry (communication with the surface station).
5. **Stage 5:** Sensor integration.
6. **Stage 6:** Camera streaming.

7. **Stage 7:** Thruster control.

8. **Stage 8:** Autonomous navigation and mission logic.

This staged approach lets you verify that the architecture works before adding hardware-specific code. It also makes debugging much easier because you can test one subsystem at a time.

Me:

For now I want to implement that when i'll run the program In the main frame The live camera will start. Before coding say what you're going to do

AI:

That's a good next milestone. I **would not jump straight into integrating every module**. Instead, I'd keep the architecture intact while implementing only the camera feature.

What I would implement

When you run:

```
python3 app/main.py
```

the following sequence would happen:

Boot

|



Load Configuration

|



Initialize Logger

|



Initialize Camera Manager

|



Open Camera



Create Main Window



Display Live Camera Feed

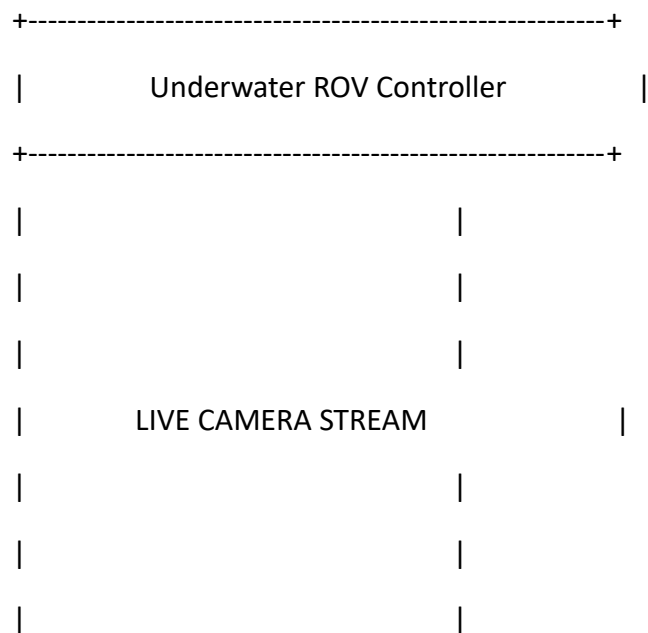


Run Event Loop

The only module with real functionality will be the **Camera Manager**. Everything else will remain placeholders.

The window

The application would open a desktop window like this:




```
+-----+
| Status: Camera Connected |
+-----+
```

Later, this same window can evolve into your complete control station.

Planned architecture

Even though only the camera works, we'll keep the modular design:

main.py

|



Application

|



CameraManager

|



OpenCV Camera

The other managers will exist but won't do anything yet:

ControllerManager (placeholder)

SensorManager (placeholder)

TelemetryManager (placeholder)

NavigationManager (placeholder)

ThrusterManager (placeholder)

MissionManager (placeholder)

DiagnosticsManager (placeholder)

WatchdogManager (placeholder)

Technologies I'd use

- **Python 3**
- **OpenCV (cv2)** for camera capture
- **Tkinter** for the GUI (built into Python, lightweight, ideal for the first version)
- **Pillow (PIL)** to display OpenCV frames inside the Tkinter window

Later, if you need a more advanced interface, we can migrate to **PySide6 (Qt)** or **Kivy**, but Tkinter is perfect for getting the camera and framework running quickly.

Folder usage

Only a few files will contain real code initially:

app/

main.py ← Starts the application

modules/

camera/

camera_manager.py ← Captures camera frames

configs/

camera.yaml ← Camera settings

Everything else stays as placeholders.

Features in Version 0.1

-  Application starts.
-  Main window opens.
-  USB camera starts automatically.

- ☒ Live video is displayed.
- ☒ Camera closes cleanly when the window exits.
- ☒ Placeholder modules initialize and print log messages.

Features reserved for later

These will stay as placeholders for now:

- Telemetry
- Sensors
- Thrusters
- Navigation
- Mission Manager
- Manipulator Arm
- Diagnostics
- Watchdog
- Data Logging

One suggestion before we code

Instead of showing **only** the camera feed, I recommend designing the window with the future layout in mind. For example:

```
+-----+
|           Underwater ROV Controller           |
+-----+-----+
|           |           |
|           |           |
|           | Live Camera Feed           |
| Left Panel |           |
```

```
| (currently empty) |  
| | |  
+-----+-----+  
| Status Bar: Camera Connected | FPS: 30 | Mode: Manual |  
+-----+
```

For now, the left panel can remain empty, but later it can hold telemetry, sensor values, mission status, and controls without requiring you to redesign the interface. This makes the GUI scalable as your ROV project grows.

Me:

So now tell me how can i setup this?

AI:

Since you're building a **professional ROV system**, I recommend setting up the project in phases instead of trying to build everything at once.

Phase 1 – Environment Setup

1. Update Raspberry Pi

```
sudo apt update
```

```
sudo apt upgrade -y
```

2. Install Python tools

```
sudo apt install python3 python3-pip python3-venv -y
```

3. Create a virtual environment

Inside your project:

```
cd ~/rov-controller
```

```
python3 -m venv .venv
```

Activate it:

```
source .venv/bin/activate
```

You should now see:

```
(.venv) theexplorer@sadman-sakib:~/rov-controller$
```

4. Install required packages

```
pip install opencv-python pillow pyyaml numpy
```

Later we'll install additional packages as needed.

Phase 2 – Configuration

Your configs/ folder already exists.

Populate the files with placeholder values, for example:

camera.yaml

camera:

device: 0

width: 1280

height: 720

fps: 30

network.yaml

network:

ip: 192.168.1.2

port: 5000

No functionality yet—just configuration.

Phase 3 – Build the Core Framework

We'll create a core/ directory to hold shared infrastructure.

mkdir core

touch core/__init__.py

touch core/application.py

touch core/base_module.py

touch core/service_manager.py

touch core/event_bus.py

These files will contain the application's foundation.

Phase 4 – Camera Module

This will be the **first real module**.

It will:

- Read camera.yaml
- Open the USB camera
- Capture frames
- Send frames to the GUI

All other modules remain placeholders.

Phase 5 – GUI

We'll use **Tkinter** for the first version.

The GUI will:

- Create the main window
- Display the live camera feed
- Show a status bar

Everything else will be empty for now.

Phase 6 – Main Application

app/main.py will:

Start

|



Load Config

|



Initialize Logger

|



Create CameraManager

|



Open GUI

|



Display Camera

|



Run Forever

Phase 7 – Automatic Startup

Once everything works, we'll create:

systemd/

 rov-controller.service

Then enable it:

`sudo systemctl enable rovd-controller`

Now, every time the Raspberry Pi boots:

Power ON

|



Raspberry Pi OS

|



systemd

|



ROV Controller

|



Main Window Opens

|



Live Camera Starts

No manual launch required.

Long-Term Roadmap

After the camera is working, we'll add modules one by one:

✓ Framework



✓ GUI



✓ Camera



✓ Logger



✓ Telemetry



✓ Sensors



✓ Thrusters



✓ Navigation



✓ Manipulator Arm



✓ Autonomous Features

My recommendation

For the GUI, I suggest **not** using Tkinter if this ROV is intended to become a serious project. Instead, use **PySide6 (Qt)** from the beginning.

Why?

- Professional-looking interface.
- Better support for multiple camera views.
- Easy integration of maps, telemetry, graphs, and controls.
- Suitable for a future surface control station.
- Used widely in robotics and industrial applications.

A good target for your first version would be:

+-----+

```

|          ROV Control Station          |
+-----+-----+
|          |          |
| Telemetry Panel |          |
| (Placeholder)  | Live Camera Feed  |
|          |          |
|          |          |
+-----+-----+
| Status: Camera Connected | FPS | Mode: Manual | Time      |
+-----+-----+

```

Only the **camera panel** will be functional initially; the telemetry panel and other controls will remain placeholders. This gives you a professional foundation that you can continue expanding without redesigning the interface later.

Me:

```
theexplorer@sadman-sakib:~/~/rov-controller $ python3 -m venv .venv
```

```
theexplorer@sadman-sakib:~/~/rov-controller $ source .venv/bin/activate
```

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ pip install opencv-python pillow pyyaml
numpy
```

Looking in indexes: <https://pypi.org/simple>, <https://www.piwheels.org/simple>

Collecting opencv-python

Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_aarch64.whl.metadata (19 kB)

Collecting pillow

Downloading pillow-12.3.0-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (9.1 kB)

Collecting pyyaml

Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl.metadata (2.4 kB)

Collecting numpy

Downloading numpy-2.5.1-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (6.6 kB)

Downloading opencv_python-5.0.0.93-cp37-abi3-manylinux_2_28_aarch64.whl (49.8 MB)

— 49.8/49.8 MB 274.6 kB/s eta 0:00:00

Downloading pillow-12.3.0-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl (6.3 MB)

— 6.3/6.3 MB 289.2 kB/s eta 0:00:00

Downloading pyyaml-6.0.3-cp313-cp313-manylinux2014_aarch64.manylinux_2_17_aarch64.manylinux_2_28_aarch64.whl (767 kB)

— 767.1/767.1 kB 374.4 kB/s eta 0:00:00

Downloading numpy-2.5.1-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl (15.2 MB)

— 15.2/15.2 MB 273.6 kB/s eta 0:00:00

Installing collected packages: pyyaml, pillow, numpy, opencv-python

Successfully installed numpy-2.5.1 opencv-python-5.0.0.93 pillow-12.3.0 pyyaml-6.0.3

(.venv) theexplorer@sadman-sakib:~/~/rov-controller \$ mkdir core

touch core/__init__.py

touch core/application.py

touch core/base_module.py

touch core/service_manager.py

touch core/event_bus.py

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ app/main.py
```

```
bash: app/main.py: Permission denied
```

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ systemd/
```

```
    rov-controller.service
```

```
bash: systemd/: Is a directory
```

```
bash: rov-controller.service: command not found
```

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $
```

What should I do in this position?

AI:

You're actually in a good position. The only issue is that you tried to **execute file names** instead of **editing or running** them.

Let's go through it.

✅ **Everything you have done so far is correct**

You successfully:

- ✅ Created the virtual environment
- ✅ Activated it
- ✅ Installed the required packages
- ✅ Created the core/ directory
- ✅ Created the core Python files

So you're ready to start coding.

Why did this happen?

You typed:

app/main.py

Bash interpreted this as:

"Run the file app/main.py as an executable."

But:

- It isn't executable (chmod +x hasn't been used).
- It doesn't have a shebang (#!/usr/bin/env python3).

That's why you got:

Permission denied

To edit the file

Use:

```
nano app/main.py
```

or, if you have VS Code installed:

```
code .
```

or:

```
code app/main.py
```

To run the Python file

Use:

```
python3 app/main.py
```

or, while your virtual environment is active:

```
python app/main.py
```

The second error

You typed:

```
systemd/
```

```
rov-controller.service
```

Those lines in my previous message were meant to illustrate the **folder structure**, not commands to execute.

To view the directory:

```
cd systemd
```

```
ls
```

To edit the service file:

```
nano systemd/rov-controller.service
```

We won't configure the actual systemd service until after the application is working.

What should you do next?

Don't touch the systemd folder yet.

The next step is to start implementing the application.

I recommend this order:

Step 1 (next)

Implement:

```
core/base_module.py
```

This becomes the parent class for every manager.

Step 2

Implement:

```
core/service_manager.py
```

This will create and manage all the modules.

Step 3

Implement:

```
core/application.py
```

This becomes the heart of the application.

Step 4

Implement:

modules/camera/camera_manager.py

This will be the **first real module**.

Step 5

Implement:

app/main.py

This starts the application.

I suggest we do this together

Instead of writing random code into each file, we'll build the framework like a real software project.

We'll start with **core/base_module.py**, because every module (Camera, Sensors, Telemetry, Navigation, etc.) will inherit from it.

Then we'll build the rest of the framework on top of it. That way, when you later add sensors, thrusters, telemetry, or autonomous navigation, the structure is already in place and consistent.

Me:

As you've seen in my last text Phase 3 has been done but after that it becomes messy and i provided the terminal command also but i dont understand what's about phase 4 and others , therefore you also have said to do more things. I also shut down the raspberry pi os then. So tell me where to start and what things you should know to continue give me commands one by one like you give me one command to do then after I've done I'll give the reply or I'll say I've done that then you'll give me another not a huge message together

AI:

Perfect. I think this is a much better way to work together.

From now on, we'll do this **like a real development session**:

- I will give you **only one step at a time**.
- You will execute it.
- You will tell me the result (or paste any error).
- Then I'll give you the next step.
- I won't overwhelm you with a long list of tasks.

We'll build the ROV project gradually.

Before we continue, here's our goal

Our first milestone is:

When you run `python3 app/main.py`, a professional window opens and displays the live camera feed.

Nothing else yet.

Step 1

First, after rebooting your Raspberry Pi, let's make sure you're back in the project and your virtual environment is active.

Run these commands:

```
cd ~/rov-controller
```

```
source .venv/bin/activate
```

```
pwd
```

Then tell me:

1. Did the prompt change to something like `(.venv)`?
2. What is the output of `pwd`?

Once you reply with the result, we'll move to Step 2.

Me:

```
theexplorer@sadman-sakib:~ $ cd ~/rov-controller
```

```
source .venv/bin/activate
```

```
pwd
```

```
bash: cd: /home/theexplorer/rov-controller: No such file or directory
```

```
bash: .venv/bin/activate: No such file or directory
```

```
/home/theexplorer
```

```
theexplorer@sadman-sakib:~ $ cd /home/theexplorer/~/rov-controller
```

```
theexplorer@sadman-sakib:~/~/rov-controller $
```

AI:

Good catch. The problem is that your project is **not** in ~/rov-controller.

It's actually inside this directory:

```
/home/theexplorer/~/rov-controller
```

That strange ~ folder is part of the path.

Step 2

Now run **only this command**:

```
source .venv/bin/activate
```

Then tell me if your prompt changes to something like:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $
```

If it does, we'll move to the next step. If it doesn't, paste the exact output.

Me:

```
yes
```

AI:

Perfect.  Your virtual environment is active.

Step 3

Let's make sure all the required Python packages are available.

Run this command:

```
pip list
```

Then **copy and paste the output here**.

After I verify it, we'll start writing the **first actual Python code**. We won't touch anything else until this is confirmed.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ pip list
```

Package	Version
-----	-----
numpy	2.5.1
opencv-python	5.0.0.93
pillow	12.3.0
pip	25.1.1
PyYAML	6.0.3

AI:

Excellent.  Everything is installed correctly.

Now we're going to write our **first real piece of code**.

Step 4

First, let's verify that your project structure is still there.

Run:

```
tree
```

Then paste the output here.

I know you shared it before, but I want to verify the current state after the reboot so we start coding against the correct structure. After that, we'll write the first Python file (core/base_module.py).

Me:

(.venv) theexplorer@sadman-sakib:~/~/rov-controller \$ tree

```
.
├── app
│   ├── app.py
│   ├── bootstrap.py
│   ├── __init__.py
│   └── main.py
├── configs
│   ├── camera.yaml
│   ├── __init__.py
│   ├── mission.yaml
│   ├── navigation.yaml
│   ├── network.yaml
│   ├── sensors.yaml
│   ├── telemetry.yaml
│   └── thrusters.yaml
├── core
│   ├── application.py
│   ├── base_module.py
│   ├── event_bus.py
│   ├── __init__.py
│   └── service_manager.py
├── __init__.py
└── logs
```

- | |— errors.log
- | |— __init__.py
- | |— mission.log
- | |— system.log
- | |— modules
- | |— camera
 - | | |— camera_manager.py
 - | | |— __init__.py
- | |— config
 - | | |— config_manager.py
 - | | |— __init__.py
- | |— controller
 - | | |— controller.py
 - | | |— __init__.py
- | |— diagnostics
 - | | |— diagnostics_manager.py
 - | | |— __init__.py
- | |— __init__.py
- | |— logger
 - | | |— __init__.py
 - | | |— logger_manager.py
- | |— manipulator
 - | | |— __init__.py
 - | | |— manipulator_manager.py
- | |— mission

- | | └─ __init__.py
- | | └─ mission_manager.py
- | └─ navigation
- | | └─ __init__.py
- | | └─ navigation_manager.py
- | └─ sensors
- | | └─ __init__.py
- | | └─ sensor_manager.py
- | └─ telemetry
- | | └─ __init__.py
- | | └─ telemetry_manager.py
- | └─ thrusters
- | | └─ __init__.py
- | | └─ thruster_manager.py
- | └─ watchdog
- | | └─ __init__.py
- | | └─ watchdog_manager.py
- └─ README.md
- └─ scripts
- | └─ __init__.py
- | └─ restart.sh
- | └─ start.sh
- | └─ stop.sh
- └─ services
- | └─ __init__.py

```
└─ systemd
|   └─ __init__.py
|       └─ rovr-controller.service
└─ tests
    └─ __init__.py
```

22 directories, 56 files

AI:

Perfect.  Your project structure is exactly as expected.

Now we'll start coding.

Step 5

Open the core/base_module.py file:

```
nano core/base_module.py
```

Don't write anything yet.

Just open the file.

When the editor opens, reply with:

Opened

Then I'll give you the code for **only this file**. We'll build the project one file at a time, keeping everything clean and easy to understand.

Me:

opened

AI:

Great. 

Now we're creating the **parent class** that every future module (Camera, Sensors, Telemetry, Navigation, etc.) will inherit from.

Copy and paste this into core/base_module.py

```
class BaseModule:
```

```
    """
```

```
    Base class for all ROV modules.
```

```
    Every module should inherit from this class.
```

```
    """
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.running = False
```

```
    def initialize(self):
```

```
        print(f"[{self.name}] Initialized")
```

```
    def start(self):
```

```
        self.running = True
```

```
        print(f"[{self.name}] Started")
```

```
    def update(self):
```

```
        if self.running:
```

```
            print(f"[{self.name}] Updating...")
```

```
    def stop(self):
```

```
        self.running = False
```

```
        print(f"[{self.name}] Stopped")
```

```
def health_check(self):  
    print(f"[{self.name}] Health: OK")
```

Save and exit Nano

1. Press **Ctrl + O** (write the file)
 2. Press **Enter**
 3. Press **Ctrl + X** (exit Nano)
-

After that, reply with:

Done

Then we'll create the **Camera Manager**, which will be the first module that uses this base class. From there, we'll begin moving toward displaying the live camera in the main window.

Me:

Done

AI:

Excellent. 🎉 That's the first real piece of your framework.

Now we'll make sure it works before writing more code.

Step 6

Open the camera manager:

```
nano modules/camera/camera_manager.py
```

Reply with:

Opened

Don't write anything yet. I want to explain this file before we add the code, because this will become the **first real module** in your ROV system.

Me:

Opened

AI:

Perfect.

Before we write the code, I want to explain **why** we're doing it this way.

Right now, we have a BaseModule. Every module in your ROV will inherit from it.

So the relationship will be:

BaseModule



|

CameraManager

SensorManager

TelemetryManager

NavigationManager

ThrusterManager

...

That means every module automatically gets:

- initialize()
- start()
- update()
- stop()
- health_check()

Now the CameraManager only needs to contain camera-specific functionality.

Step 6 Code

Replace the contents of modules/camera/camera_manager.py with this:

```
from core.base_module import BaseModule
```

```
class CameraManager(BaseModule):  
    def __init__(self):  
        super().__init__("Camera")  
  
    def initialize(self):  
        super().initialize()  
        print("[Camera] Waiting for camera implementation...")  
  
    def update(self):  
        if self.running:  
            print("[Camera] Camera is running...")
```

Save it:

- **Ctrl + O**
- **Enter**
- **Ctrl + X**

Then reply only:

Done

After that, we'll make app/main.py create and run the CameraManager. That will be the first time your framework actually executes code.

Me:

Done

AI:

Excellent. You're doing great.

Now we'll write our **first runnable program**. When we run it, it will initialize and start the CameraManager. This verifies that our framework is working before we add the real camera.

Step 7

Open the main file:

```
nano app/main.py
```

Reply with:

Opened

Don't type anything yet. I'll give you the code next, and then we'll run your project for the first time.

Me:

Opened

AI:

Great. Now we're going to create the **first runnable version** of your ROV application.

Replace everything in **app/main.py** with this code:

```
from modules.camera.camera_manager import CameraManager
```

```
def main():
```

```
    print("=" * 50)
```

```
    print("    Underwater ROV Controller v0.1")
```

```
    print("=" * 50)
```

```
    # Create the Camera Manager
```

```
    camera = CameraManager()
```

```
    # Initialize the module
```

```
camera.initialize()
```

```
# Start the module
```

```
camera.start()
```

```
# Run one update
```

```
camera.update()
```

```
print("=" * 50)
```

```
print("Framework started successfully!")
```

```
print("=" * 50)
```

```
if __name__ == "__main__":
```

```
    main()
```

Then:

1. **Save** (Ctrl + O, then Enter)
2. **Exit** (Ctrl + X)

Next

Run exactly this command:

```
python app/main.py
```

✂ **Copy and paste the entire output here.**

Don't try to fix any errors yourself if one appears. If there's an error, we'll debug it together step by step.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ python app/main.py
```

Traceback (most recent call last):

```
File "/home/theexplorer/~/rov-controller/app/main.py", line 1, in <module>
```

```
    from modules.camera.camera_manager import CameraManager
```

ModuleNotFoundError: No module named 'modules'

AI:

Excellent! This is a **normal and expected** error. We're not stuck—this is the first thing you usually encounter in a modular Python project.

The reason is:

When you run:

```
python app/main.py
```

Python thinks the app folder is the starting point, so it doesn't know about the sibling folder modules.

We'll fix this properly.

Step 8

Open app/main.py again:

```
nano app/main.py
```

Reply with:

Opened

I'll show you the correct way to make Python find your project modules. We'll use a solution that's appropriate for your project rather than a quick workaround.

Me:

Opened

AI:

Good. We're going to make **one small change**.

At the **very top** of app/main.py, before any other imports, add these lines:

```
import os
```

```
import sys
```

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
sys.path.insert(0, PROJECT_ROOT)
```

So the beginning of your file should look like this:

```
import os
```

```
import sys
```

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
```

```
sys.path.insert(0, PROJECT_ROOT)
```

```
from modules.camera.camera_manager import CameraManager
```

Leave the rest of the file exactly as it is.

Then:

1. **Save** (Ctrl + O, Enter)

2. **Exit** (Ctrl + X)

Finally, run:

```
python app/main.py
```

Copy and paste the entire output here.

Note: This is a temporary solution to get us moving. Later, we'll switch to a cleaner package-based approach (for example, running the project as `python -m app.main`), which is the recommended way for production code. For now, this lets us focus on building the ROV framework.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ python app/main.py
```

```
File "/home/theexplorer/~/rov-controller/app/main.py", line 12
```

```
    print("    Underwater ROV
```

```
    ^
```

```
SyntaxError: unterminated string literal (detected at line 12)
```

AI:

Perfect! This is **not** an import problem anymore. 

The new error means your main.py has a **typing or copy-paste mistake**.

The error:

```
SyntaxError: unterminated string literal
```

means one of your `print("...")` lines is missing the closing quotation mark (`"`), or the string was accidentally split across two lines.

Step 9

Let's inspect the file instead of guessing.

Run this command:

```
cat app/main.py
```

Then **copy and paste the entire contents of app/main.py here**.

Don't edit it yourself yet. I'll identify the exact problem and tell you what to change. This way we avoid introducing more errors.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ cat app/main.py
```

```
import os
```

```
import sys
```

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
sys.path.insert(0, PROJECT_ROOT)
```

```
from modules.camera.camera_manager import CameraManager
```

```
def main():
```

```
    print("=" * 50)
```

```
    print("    Underwater ROV
```

```
Controller v0.1")
```

```
    print("=" * 50)
```

```
    # Create the Camera Manager
```

```
    camera = CameraManager()
```

```
    # Initialize the module
```

```
    camera.initialize()
```

```
    # Start the module
```

```
    camera.start()
```

```
    # Run one update
```

```
    camera.update()
```

```
    print("=" * 50)
```



```
print("Framework started successfully!")  
print("=" * 50)
```

```
if __name__ == "__main__":  
    main()
```

AI:

I found the problem immediately. 👍

This line got split into **two lines** when you pasted it:

```
print("    Underwater ROV
```

```
Controller v0.1")
```

Python doesn't allow a normal string to span multiple lines like that.

Step 10

Open the file again:

```
nano app/main.py
```

Replace these two broken lines:

```
print("    Underwater ROV
```

```
Controller v0.1")
```

with this **single line**:

```
print("    Underwater ROV Controller v0.1")
```

So the beginning of main() should look like this:

```
def main():
```

```
    print("=" * 50)
```

```
print("    Underwater ROV Controller v0.1")  
print("=" * 50)
```

Then:

1. Save (Ctrl + O, Enter)
2. Exit (Ctrl + X)
3. Run:

```
python app/main.py
```

Paste the output here.

Small note

From now on, whenever I give you Python code, **copy the whole code block** rather than typing it manually. That will prevent small syntax errors like this one and make the process much smoother.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ python app/main.py
```

```
=====
```

```
    Underwater ROV Controller v0.1
```

```
=====
```

```
[Camera] Initialized
```

```
[Camera] Waiting for camera implementation...
```

```
[Camera] Started
```

```
[Camera] Camera is running...
```

```
=====
```

```
Framework started successfully!
```

```
=====
```

AI:

Excellent! 🎉

This is your **first working version** of the ROV framework.

Let's pause and recognize what you've built:

- ✅ main.py starts correctly.
- ✅ CameraManager is created.
- ✅ CameraManager inherits from BaseModule.
- ✅ The framework initializes, starts, and updates the module.
- ✅ Your project architecture is working.

This is exactly the kind of foundation we wanted before adding hardware.

Next Milestone

Now we're going to stop printing:

[Camera] Camera is running...

and replace it with a **real live camera feed**.

So when you run:

```
python app/main.py
```

you'll get a window like this:

```
+-----+
|      Underwater ROV Controller v0.1      |
+-----+
|                                           |
|                                           |
|      LIVE CAMERA FEED                    |
|                                           |
|                                           |
```

```
+-----+
| Status: Camera Connected |
+-----+
```

Before we code the camera

I want to make one design decision now because it affects the whole project.

Option A — OpenCV Window (Very Easy)

The camera opens in a standard OpenCV window.

```
cv2.imshow(...)
```

Pros:

- Very simple
- Fast to implement
- Good for testing

Cons:

- Not suitable for a professional control station
 - Difficult to add telemetry, buttons, maps, and status panels later
-

Option B — GUI Window (Recommended)

Create a real application window and display the camera inside it.

Pros:

- Professional architecture
- Easy to add telemetry, buttons, sensor panels, mission controls, and maps
- This becomes your final ROV control station

Cons:

- Slightly more work now, but much better long-term
-

My recommendation

Since this is your **main ROV project**, I recommend **Option B**.

We'll build the application as a real control station from the beginning. Initially, only the camera will work, and the rest of the interface will be placeholders.

That way, you won't need to redesign everything later.

The window will gradually evolve like this:

Version 0.1

+-----+	
	Underwater ROV
+-----+	
	Live Camera Feed
+-----+	
	Status: Connected
+-----+	

Future Version

+-----+	
	Underwater ROV Control Station
+-----+	
	Telemetry
	Depth
	IMU
	Battery
	Leak Status
+-----+	

| Thrusters | Mission | Diagnostics | Logs | Settings | Exit |

+-----+

I strongly recommend we follow this path because it gives you a scalable, professional interface without needing to rebuild the GUI later.

Me:

ok let's forward

AI:

Great! 🚀 We'll continue one step at a time.

Step 11

Since we're building a **professional GUI**, we'll use **PySide6 (Qt)** instead of Tkinter.

Install it in your virtual environment:

```
pip install PySide6
```

⚠️ **Note:** PySide6 is a large package, so it may take several minutes to download and install on a Raspberry Pi.

When it's finished, don't do anything else.

Just reply with either:

- **Done** (if it installed successfully), or
- Paste the error message if something went wrong.

Then we'll create your **first application window** before adding the live camera feed.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ pip install PySide6
```

Looking in indexes: <https://pypi.org/simple>, <https://www.piwheels.org/simple>

Collecting PySide6

Downloading pyside6-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl.metadata (5.3 kB)

Collecting shiboken6==6.11.1 (from PySide6)

Downloading shiboken6-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl.metadata (2.4 kB)

Collecting PySide6_Essentials==6.11.1 (from PySide6)

Downloading pyside6_essentials-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl.metadata (3.6 kB)

Collecting PySide6_Addons==6.11.1 (from PySide6)

Downloading pyside6_addons-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl.metadata (4.0 kB)

Downloading pyside6-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl (572 kB)

_____ 572.1/572.1 kB 2.8 MB/s eta
0:00:00

Downloading pyside6_addons-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl (170.6 MB)

_____ 170.6/170.6 MB 505.6 kB/s eta
0:00:00

Downloading pyside6_essentials-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl (79.0 MB)

_____ 79.0/79.0 MB 1.1 MB/s eta
0:00:00

Downloading shiboken6-6.11.1-cp310-abi3-manylinux_2_39_aarch64.whl (270 kB)

Installing collected packages: shiboken6, PySide6_Essentials, PySide6_Addons, PySide6

Successfully installed PySide6-6.11.1 PySide6_Addons-6.11.1 PySide6_Essentials-6.11.1
shiboken6-6.11.1

AI:

Perfect.  PySide6 installed successfully.

From this point on, we're going to stop using the terminal as the "interface." Everything will be displayed in a proper GUI window.

Step 12

First, create a folder for the GUI.

Run this command:

```
mkdir -p ui
```

```
touch ui/__init__.py
```

```
touch ui/main_window.py
```

After that, run:

```
tree ui
```

You should see something like:

```
ui
```

```
├── __init__.py
```

```
└── main_window.py
```

Reply with the output of tree ui.

Why are we creating a ui folder?

To keep the project organized:

- app/ → Starts the application.
- core/ → Framework and shared classes.
- modules/ → Camera, sensors, telemetry, etc.
- ui/ → All graphical user interface code.

This separation will make the project much easier to maintain as it grows.

Me:

```
(.venv) theexplorer@sadman-sakib:~/~/rov-controller $ tree ui
```

```
ui
```

```
├── __init__.py
```

```
└── main_window.py
```

1 directory, 2 files

AI:

Excellent. 

Your project is becoming well organized.

Now we'll create the **main application window**. **Don't worry about the camera yet**. First, we want to confirm that PySide6 works and that we can open a window.

Step 13

Open the file:

nano ui/main_window.py

Reply with:

Opened

Don't type anything yet.

I'll give you the code for this file, and after that, we'll modify app/main.py to launch the GUI.

Once we see the window successfully, the **next step** will be embedding the live camera feed into it. This incremental approach makes debugging much easier.

Me:

Opened